



Universidade Federal do Ceará

Campus Quixadá

QXD0099 - Desenvolvimento de Software para Persistência

Prof. Francisco Victor da Silva Pinheiro

***Trabalho Prático 1 - Desenvolvimento de um Cofre Digital de Arquivos com FastAPI
Persistência, Serialização, Integridade, Segurança e Backup***

1. Descrição Geral

Neste Trabalho Prático 1, os alunos deverão desenvolver, utilizando Python e FastAPI, uma aplicação denominada Cofre Digital de Arquivos. O sistema deverá permitir o armazenamento, gerenciamento, consulta, proteção, verificação de integridade, exportação e recuperação de arquivos digitais, mantendo também seus metadados. A aplicação deverá integrar diferentes formatos estudados no Módulo 1, como JSON, XML, CSV, YAML, arquivos de propriedades, logs e arquivos binários, aplicando-os de forma coerente no sistema. Todas as equipes desenvolverão a mesma estrutura geral, porém cada uma trabalhará com um domínio específico, contendo metadados, consultas e funcionalidades próprias relacionadas ao tema recebido.

2. Cenário da Aplicação

Imagine uma organização que necessita armazenar documentos digitais de maneira organizada e controlada. Esses documentos podem ser, por exemplo: contratos; relatórios; notas fiscais; comprovantes; documentos administrativos; imagens; arquivos de texto; planilhas; arquivos PDF; documentos acadêmicos; documentos pessoais; manuais; e outros arquivos digitais. A organização deseja possuir um pequeno sistema denominado **Cofre Digital**, responsável por armazenar esses arquivos e manter informações sobre cada documento.

Ao enviar um documento para o sistema, a aplicação deverá:

1. receber o arquivo pela API;
2. armazenar o arquivo no sistema de arquivos;
3. identificar suas principais características;
4. gerar um identificador único;
5. calcular seu hash SHA-256;
6. registrar seus metadados;
7. registrar a operação no arquivo de log.

A partir desse momento, o documento poderá ser consultado, baixado, atualizado, verificado, exportado ou incluído em backups.

3. Organização Geral dos Dados

O sistema deverá manter separadamente os arquivos originais; os metadados dos documentos; os arquivos de configuração; os logs da aplicação; os arquivos exportados; os backups gerados.

Uma possível organização é:

A equipe poderá propor outra estrutura, desde que seja organizada e permita identificar claramente a responsabilidade de cada arquivo e diretório.

4. Modelo de Documento

Cada arquivo armazenado deverá possuir um conjunto de metadados. A equipe deverá implementar uma classe utilizando **Pydantic** para representar os documentos cadastrados.

O modelo deverá possuir, no mínimo:

- id;
- nome_original;
- nome_armazenado;
- extensao;
- tipo_mime;
- tamanho;
- categoria;
- descricao;
- data_upload;
- sha256.

Exemplo:

```
{
  "id": 15,
  "nome_original": "contrato_cliente.pdf",
  "nome_armazenado": "15_contrato_cliente.pdf",
  "extensao": ".pdf",
  "tipo_mime": "application/pdf",
  "tamanho": 458921,
  "categoria": "contrato",
  "descricao": "Contrato de prestação de serviços",
  "data_upload": "2026-09-10T14:32:18",
  "sha256": "34ab72..."
}
```

Além desses campos comuns, cada equipe deverá acrescentar ao modelo os **metadados específicos relacionados ao tema recebido**.

5. Persistência Principal dos Metadados

Os metadados dos documentos deverão ser persistidos em **JSON**. Por exemplo: storage/metadata/documentos.json. O arquivo deverá representar os documentos atualmente cadastrados no sistema.

Exemplo:

```
[
  {
    "id": 1,
    "nome_original": "relatorio.pdf",
    "categoria": "relatorio",
    "sha256": "...",
  },
  {
    "id": 2,
    "nome_original": "contrato.pdf",
    "categoria": "contrato",
    "sha256": "...",
  }
]
```

A aplicação deverá efetivamente ler e escrever nesse arquivo. Não será aceita uma solução na qual os dados permaneçam apenas em listas ou variáveis em memória. Quando a aplicação for encerrada e iniciada novamente, os documentos cadastrados anteriormente deverão continuar disponíveis.

6. Funcionalidades Obrigatórias

F1 — Upload e armazenamento de arquivos

Deverá existir um endpoint responsável pelo recebimento de arquivos. Exemplo: POST /documentos. O endpoint deverá permitir enviar um arquivo juntamente com seus principais metadados, incluindo aqueles específicos do domínio.

Quando o arquivo for enviado, o sistema deverá:

1. gerar um identificador para o documento;
2. preservar o nome original;
3. definir um nome adequado para armazenamento;
4. armazenar fisicamente o arquivo;
5. identificar sua extensão;
6. identificar, quando possível, seu tipo MIME;
7. calcular seu tamanho;
8. calcular seu hash SHA-256;
9. registrar os metadados no JSON;
10. registrar a operação no log.

O sistema deverá evitar que arquivos diferentes sejam sobrescritos simplesmente porque possuem o mesmo nome.

F2 — Listagem de documentos

Deverá existir um endpoint: GET /documentos. Esse endpoint deverá retornar todos os documentos armazenados no sistema. Os dados deverão ser recuperados a partir do arquivo de persistência.

Exemplo de resposta:

```
[
  {
    "id": 1,
    "nome_original": "contrato.pdf",
    "categoria": "contrato",
    "tamanho": 332012
  },
  {
    "id": 2,
    "nome_original": "relatorio.pdf",
    "categoria": "relatorio",
    "tamanho": 125891
  }
]
```

F3 — Consulta de documento por identificador

Deverá existir: GET /documentos/{id}. O endpoint deverá retornar os metadados correspondentes ao documento solicitado. Caso o documento não exista, deverá ser retornada uma resposta HTTP adequada. Exemplo: 404 — Documento não encontrado

F4 — Download do arquivo

Deverá existir: GET /documentos/{id}/download. Esse endpoint deverá permitir recuperar o **arquivo original armazenado pelo sistema**. A aplicação deverá trabalhar corretamente com arquivos binários, evitando qualquer alteração no conteúdo durante o armazenamento ou download. O arquivo baixado deverá ser equivalente ao arquivo originalmente enviado.

F5 — Atualização de metadados

Deverá existir uma operação para atualização dos metadados do documento.
Exemplo: PUT /documentos/{id}.

Poderão ser alteradas informações como:

- categoria;
- descrição;
- metadados específicos do tema;
- outros metadados definidos pela equipe.

A atualização de metadados não deverá alterar indevidamente o arquivo físico. A alteração deverá ser persistida no JSON e registrada no log.

F6 — Exclusão de documentos

Deverá existir: DELETE /documentos/{id}. A exclusão deverá manter a consistência entre o armazenamento físico e os metadados. A equipe deverá definir e documentar o comportamento adotado. No mínimo, deverão ser removidos: o registro correspondente do arquivo JSON; o arquivo armazenado correspondente. A operação deverá ser registrada no log.

F7 — Filtragem de documentos

A API deverá permitir a realização de consultas utilizando pelo menos **três critérios diferentes**. Deverá existir: pelo menos um filtro utilizando um atributo geral; pelo menos dois filtros utilizando informações específicas do domínio da equipe.

Exemplos gerais:

- GET /documentos?categoria=contrato
- GET /documentos?extensao=pdf
- Dependendo do tema, poderão existir consultas como:
- GET /documentos?numero_processo=12345
- GET /documentos?animal=Rex
- GET /documentos?projeto=ProjetoA
- GET /documentos?autor=Machado

Também poderão ser implementados filtros por:

- nome;
- tipo MIME;
- tamanho;
- data;
- categoria;
- extensão;
- informações específicas do domínio.

É permitido combinar filtros.

F8 — Estatísticas do Cofre Digital

Deverá existir: GET /documentos/estatisticas. O endpoint deverá calcular informações utilizando os dados realmente persistidos.

Deverá retornar, no mínimo: quantidade total de documentos; tamanho total ocupado; quantidade por extensão; quantidade por categoria; pelo menos uma estatística específica relacionada ao domínio da equipe.

Exemplo:

```
{
  "total_documentos": 20,
  "espaco_utilizado_bytes": 12598003,
  "por_extensao": {
    "pdf": 8,
    "txt": 4,
    "jpg": 5,
    "csv": 3
  },
  "por_categoria": {
    "contrato": 7,
    "relatorio": 8,
    "outros": 5
  }
}
```

F9 — Verificação de integridade

Cada arquivo deverá possuir um hash **SHA-256** calculado no momento do upload. Deverá existir: GET /documentos/{id}/integridade

Ao acessar esse endpoint, o sistema deverá: localizar fisicamente o documento; recalcular o hash SHA-256 do arquivo atual; recuperar o hash armazenado no momento do upload; comparar os dois valores; informar se o documento continua íntegro.

Exemplo:

```
{
  "id": 10,
  "nome": "contrato.pdf",
  "hash_original": "abc123...",
  "hash_atual": "abc123...",
  "integro": true
}
```

Caso alguém altere manualmente o arquivo armazenado, o sistema deverá ser capaz de identificar a alteração.

Exemplo:

```
{
  "id": 10,
  "nome": "contrato.pdf",
  "hash_original": "abc123...",
  "hash_atual": "f45ad1...",
  "integro": false
}
```

F10 — Verificação global de integridade

Além da verificação individual, deverá existir: GET /integridade. Esse endpoint deverá verificar todos os documentos atualmente cadastrados.

Deverá informar:

- quantidade de documentos verificados;
- quantidade de documentos íntegros;
- quantidade de documentos alterados;
- arquivos cadastrados cujo arquivo físico não foi localizado.

F11 — Sistema de Logging

A aplicação deverá utilizar a API de **logging do Python**. O arquivo deverá ser armazenado, por exemplo, em: storage/logs/sistema.log

Deverão ser registrados eventos como:

- inicialização do sistema;
- upload;
- download;
- consulta;
- atualização;
- exclusão;
- criação de backup;

- verificação de integridade;
- tentativa de acessar documento inexistente;
- erros de leitura ou escrita;
- operações de criptografia;
- operações de recuperação de backup.

Cada registro deverá possuir, no mínimo:

- data;
- hora;
- nível;
- operação;
- informações relevantes.

Exemplo:

```
2026-09-10 14:32:18 INFO UPLOAD id=10 arquivo=contrato.pdf
2026-09-10 14:35:09 INFO DOWNLOAD id=10 arquivo=contrato.pdf
2026-09-10 14:38:41 WARNING INTEGRIDADE_FALHOU id=10
2026-09-10 14:42:15 ERROR DOCUMENTO_NAO_ENCONTRADO id=50
```

Os níveis INFO, WARNING e ERROR deverão ser utilizados de maneira coerente.

F12 — Arquivo de Configuração

A aplicação deverá possuir um arquivo externo de configuração utilizando: YAML; ou, arquivo [.properties](#).

Exemplo:

```
storage:
  diretorio_documentos: "./storage/documentos"
  diretorio_backups: "./storage/backups"

upload:
  tamanho_maximo_mb: 20

hash:
  algoritmo: "sha256"

logging:
  arquivo: "./storage/logs/sistema.log"
  nivel: "INFO"

backup:
  formato: "zip"
```

A aplicação deverá **ler e utilizar efetivamente essas configurações**. Não será considerado suficiente apenas criar o arquivo sem integrá-lo ao funcionamento da aplicação. Por exemplo, alterar o nível de logging ou determinado diretório no arquivo de configuração deverá produzir efeito no sistema.

F13 — Exportação para CSV

Deverá existir: GET /exportar/csv O endpoint deverá gerar um arquivo contendo o catálogo atual dos documentos.

Exemplo:

```
id,nome_original,extensao,categoria,tamanho,data_upload,sha256
1,contrato.pdf,.pdf,contrato,500230,2026-09-10T14:30:00,...
2,relatorio.txt,.txt,relatorio,8500,2026-09-10T14:35:00,...
```

Os metadados específicos relacionados ao domínio também deverão ser considerados na exportação quando forem relevantes. O arquivo deverá ser gerado a partir dos dados persistidos no sistema.

F14 — Backup compactado

Deverá existir: POST /backup. Essa operação deverá gerar um arquivo de backup compactado. O arquivo poderá ser gerado utilizando, por exemplo: backup_2026-09-25_1430.zip O backup deverá ser armazenado em: storage/backups/ ou no diretório definido pelo arquivo de configuração. O sistema deverá evitar a sobrescrita indevida de backups anteriores. Nos temas em que houver **backup seletivo como requisito específico**, além do backup geral deverá existir a possibilidade de selecionar os documentos de acordo com o domínio.

F15 — Listagem de backups

Deverá existir: GET /backups. A aplicação deverá retornar os backups disponíveis.

Exemplo:

```
[
  {
    "arquivo": "backup_2026-09-20_1400.zip",
    "tamanho": 10548922
  },
  {
    "arquivo": "backup_2026-09-25_1430.zip",
    "tamanho": 11850340
  }
]
```

F16 — Funcionalidade Específica do Tema

Cada equipe deverá implementar obrigatoriamente a **funcionalidade específica associada ao tema recebido**, conforme descrito.

Essa funcionalidade deverá:

- estar integrada à API;
- utilizar os dados efetivamente persistidos;

- utilizar pelo menos um dos metadados específicos do domínio;
- possuir endpoint próprio quando aplicável;
- estar documentada no README;
- ser demonstrada durante a apresentação.

Não será considerada suficiente uma funcionalidade simulada ou que trabalhe apenas com dados inseridos diretamente no código.

F17. Tratamento de Erros

A aplicação deverá tratar adequadamente situações como: documento inexistente; arquivo físico não encontrado; JSON inválido; XML inválido; arquivo de configuração inexistente ou inválido; tentativa de upload inválida; falha durante compactação; backup inexistente; problemas durante criptografia; tentativa de descriptografia com chave incorreta; erro de leitura ou escrita. A API deverá utilizar códigos HTTP adequados sempre que possível.

Exemplos:

- 200 — operação realizada com sucesso;
- 201 — recurso criado;
- 400 — requisição inválida;
- 404 — recurso não encontrado;
- 409 — conflito;
- 422 — dados inválidos;
- 500 — erro interno inesperado.

7. Dados para Apresentação

Antes da apresentação, o sistema deverá possuir dados previamente cadastrados. O Cofre Digital deverá possuir, no mínimo: 15 arquivos armazenados; pelo menos 4 extensões diferentes; pelo menos 3 categorias diferentes; arquivos de texto; pelo menos um arquivo binário; metadados adequadamente persistidos; dados representativos do domínio recebido. Não será permitido iniciar a apresentação com o sistema completamente vazio.

8. README

O projeto deverá possuir um arquivo: README.md contendo, no mínimo:

- nome do projeto;
- integrantes;
- tema recebido;
- objetivo;
- requisitos;
- bibliotecas utilizadas;
- instruções de instalação;
- instruções de execução;
- descrição da estrutura do projeto;
- principais endpoints;
- exemplos de utilização;

- metadados específicos do domínio;
- descrição da funcionalidade específica do tema.

Uma pessoa que receba o projeto deverá conseguir executá-lo seguindo as instruções fornecidas.

9. Restrições

- não deverá ser utilizado banco de dados para substituir os mecanismos de persistência solicitados;
- os metadados principais deverão ser persistidos em arquivo;
- JSON, XML, CSV, configuração externa e logs deverão ser efetivamente utilizados;
- as funcionalidades de persistência não poderão depender exclusivamente de variáveis em memória;
- os metadados específicos do domínio deverão ser utilizados efetivamente;
- a funcionalidade específica do tema deverá estar integrada ao restante da aplicação;
- o uso de bibliotecas é permitido, desde que a equipe compreenda e consiga explicar seu funcionamento;
- não será considerado suficiente implementar funções que apenas criem arquivos fictícios sem integração com o sistema;
- não será considerada adequada a simples adaptação superficial de um sistema desenvolvido para outro tema.

"A melhor maneira de prever o futuro é inventá-lo."
— Alan Kay

Temas

Equipe	Tema do Cofre Digital	Tipos de arquivos esperados	Particularidade obrigatória
1	Prontuários Veterinários	PDF, imagens, exames, receitas	Metadados de animal, tutor, espécie e data do atendimento
2	Documentos Acadêmicos	PDF, DOCX, certificados, históricos	Categoria acadêmica, aluno, curso e semestre
3	Contratos Empresariais	PDF, DOCX, anexos	Data de início, validade, contratante e contratado
4	Documentos Jurídicos	PDF, imagens, petições	Número do processo, tipo do documento e partes envolvidas
5	Acervo Fotográfico Institucional	JPG, PNG, TIFF	Evento, data, autor e local da fotografia
6	Documentação de Projetos de Software	PDF, MD, JSON, YAML, diagramas	Projeto, versão, tipo de artefato e responsável
7	Documentos de Recursos Humanos	PDF, comprovantes, formulários	Funcionário, setor, tipo e competência
8	Documentação de Patrimônio	PDF, imagens, notas fiscais	Número patrimonial, setor, categoria e situação
9	Acervo de Pesquisa Científica	PDF, CSV, JSON, imagens	Projeto, pesquisador, tipo de artefato e data de coleta
10	Documentos Financeiros	PDF, CSV, XML, comprovantes	Tipo, competência, valor e centro de custo
11	Documentação de Eventos	PDF, imagens, certificados	Evento, participante, categoria e data
12	Acervo de Obras e Engenharia	PDF, imagens, plantas, planilhas	Obra, etapa, responsável técnico e versão
13	Arquivos de Segurança da Informação	LOG, TXT, JSON, CSV, relatórios	Origem, severidade, tipo de evento e data/hora
14	Documentação de Laboratórios	PDF, CSV, imagens, relatórios	Laboratório, equipamento, experimento e responsável
15	Biblioteca Digital	PDF, EPUB, TXT, imagens	Autor, título, gênero/categoria e ano

Descrição dos temas

Tema 1 — Cofre de Prontuários Veterinários

O sistema deverá armazenar documentos relacionados ao atendimento de animais, como:

- prontuários;
- exames;
- receitas;
- laudos;
- imagens;
- comprovantes de vacinação.

Além dos metadados gerais, deverão existir informações como:

- nome do animal;
- espécie;
- raça;
- nome do tutor;
- data do atendimento.

Requisito específico: O sistema deverá permitir recuperar todos os documentos relacionados a determinado animal e gerar um **backup seletivo** contendo apenas os documentos desse animal.

Tema 2 — Cofre de Documentos Acadêmicos

O sistema deverá armazenar documentos relacionados à vida acadêmica de estudantes, como:

- históricos;
- certificados;
- declarações;
- comprovantes;
- trabalhos;
- documentos de matrícula.

Além dos metadados gerais, deverão existir informações como:

- aluno;
- matrícula;
- curso;
- semestre;
- tipo do documento.

Requisito específico: O sistema deverá permitir consultar documentos por **aluno ou semestre** e gerar uma exportação XML contendo os documentos selecionados.

Tema 3 — Cofre de Contratos Empresariais

O sistema deverá armazenar contratos e documentos relacionados às relações comerciais de uma organização.

Além dos metadados gerais, deverão existir informações como:

- contratante;
- contratado;

- data de início;
- data de término;
- situação do contrato.

Requisito específico

O sistema deverá possuir uma consulta capaz de identificar:

- contratos vencidos;
- contratos vigentes;
- contratos próximos da data de vencimento.

Tema 4 — Cofre de Documentos Jurídicos

O sistema deverá armazenar documentos relacionados a processos jurídicos, como:

- petições;
- pareceres;
- decisões;
- procurações;
- comprovantes;
- anexos.

Além dos metadados gerais, deverão existir informações como:

- número do processo;
- tipo do documento;
- parte envolvida;
- advogado responsável;
- data do documento.

Requisito específico: O sistema deverá permitir recuperar e exportar todos os documentos pertencentes a determinado **processo jurídico**.

Tema 5 — Acervo Fotográfico Institucional

O sistema deverá armazenar fotografias e outros arquivos relacionados ao registro de atividades institucionais.

Além dos metadados gerais, deverão existir informações como:

- evento;
- autor ou fotógrafo;
- data;
- local;
- descrição da imagem.

Requisito específico

O sistema deverá apresentar estatísticas do acervo considerando:

- evento;
- ano;
- formato da imagem.

Tema 6 — Cofre de Documentação de Projetos de Software

O sistema deverá armazenar artefatos relacionados ao desenvolvimento de projetos de software, como:

- requisitos;
- documentação;
- diagramas;
- arquivos Markdown;
- configurações;
- especificações;
- relatórios.

Além dos metadados gerais, deverão existir informações como:

- projeto;
- versão;
- tipo de artefato;
- responsável;
- data da versão.

Requisito específico O sistema deverá permitir recuperar todos os documentos relacionados a uma determinada **versão ou release** de um projeto.

Tema 7 — Cofre de Documentos de Recursos Humanos

O sistema deverá armazenar documentos relacionados aos funcionários de uma organização.

Além dos metadados gerais, deverão existir informações como:

- funcionário;
- setor;
- tipo do documento;
- competência;
- data.

Requisito específico: O sistema deverá permitir gerar um arquivo **ZIP contendo todos os documentos relacionados a determinado funcionário**.

Tema 8 — Cofre de Documentação Patrimonial

O sistema deverá armazenar documentos relacionados aos bens patrimoniais de uma organização.

Além dos metadados gerais, deverão existir informações como:

- número patrimonial;
- equipamento ou bem;
- setor;
- situação;
- responsável.

Requisito específico: O sistema deverá permitir recuperar todos os documentos relacionados a determinado **número patrimonial**.

Tema 9 — Cofre de Artefatos de Pesquisa Científica

O sistema deverá armazenar materiais relacionados a projetos de pesquisa, como:

- artigos;
- datasets;
- formulários;
- relatórios;
- imagens;
- scripts;
- resultados experimentais.

Além dos metadados gerais, deverão existir informações como:

- projeto;
- pesquisador;
- tipo de artefato;
- etapa da pesquisa;
- data.

Requisito específico: O sistema deverá permitir gerar um **backup seletivo contendo todos os artefatos pertencentes a determinado projeto de pesquisa**.

Tema 10 — Cofre de Documentos Financeiros

O sistema deverá armazenar documentos relacionados à gestão financeira, como:

- notas fiscais;
- recibos;
- comprovantes;
- relatórios financeiros;
- planilhas;
- documentos de pagamento.

Além dos metadados gerais, deverão existir informações como:

- tipo do documento;
- competência;
- valor;
- centro de custo;
- responsável.

Requisito específico

O sistema deverá apresentar estatísticas dos documentos considerando:

- competência;
- categoria;
- centro de custo.

Tema 11 — Cofre de Documentação de Eventos

O sistema deverá armazenar documentos produzidos ou utilizados na organização de eventos.

Além dos metadados gerais, deverão existir informações como:

- evento;
- participante ou responsável;
- categoria;
- data;
- local.

Requisito específico: O sistema deverá permitir gerar um arquivo **XML contendo todos os documentos associados a determinado evento**.

Tema 12 — Cofre de Documentos de Obras e Engenharia

O sistema deverá armazenar documentos associados a obras e projetos de engenharia, como:

- plantas;
- relatórios;
- memoriais;
- fotografias;
- planilhas;
- documentos técnicos.

Além dos metadados gerais, deverão existir informações como:

- obra;
- etapa;
- responsável técnico;
- versão;
- data.

Requisito específico: O sistema deverá permitir identificar e recuperar a **versão mais recente de determinado documento**.

Tema 13 — Cofre de Arquivos de Segurança da Informação

O sistema deverá armazenar arquivos relacionados ao monitoramento e análise de segurança, como:

- logs;
- relatórios;
- arquivos CSV;
- JSON;
- evidências;
- resultados de ferramentas.

Além dos metadados gerais, deverão existir informações como:

- origem;
- severidade;
- tipo de evento;
- data e hora;
- sistema de origem.

Requisito específico

O sistema deverá aceitar arquivos **.log** e permitir consultas por:

- nível de severidade;

- tipo de evento;
- sistema de origem.

Tema 14 — Cofre de Documentação de Laboratórios

O sistema deverá armazenar documentos relacionados às atividades de laboratórios, como:

- experimentos;
- relatórios;
- resultados;
- imagens;
- planilhas;
- manuais de equipamentos.

Além dos metadados gerais, deverão existir informações como:

- laboratório;
- equipamento;
- experimento;
- responsável;
- data.

Requisito específico: O sistema deverá permitir gerar uma exportação ou backup contendo apenas os documentos relacionados a determinado **equipamento ou experimento**.

Tema 15 — Biblioteca Digital

O sistema deverá armazenar materiais bibliográficos digitais, como:

- livros;
- artigos;
- apostilas;
- documentos PDF;
- arquivos EPUB;
- arquivos TXT.

Além dos metadados gerais, deverão existir informações como:

- título;
- autor;
- ano;
- gênero ou categoria;
- editora.

Requisito específico

O sistema deverá permitir realizar buscas combinadas utilizando:

- autor;
- ano;
- categoria;
- formato do arquivo.